This technique works fine as long as you're only dealing with helper methods. Sooner or later, though, you'll find that you want to share some let declarations or hooks instead.

For example, you may want to add authentication to your API. After you've done so, you'll need to modify your existing specs to log in before they make their requests. Since being logged in is an extraneous detail for most of your specs, a before hook would be the perfect place to put this new behavior:

07-structuring-code-examples/09/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb

```
before do
  basic_authorize 'test_user', 'test_password'
end
```

Here, we're using the basic_authorize method from Rack::Test to treat the HTTP request as coming from a logged-in test user.

This hook can't go into your APIHelpers module, though. Plain Ruby modules aren't aware of RSpec constructs such as hooks. Instead, you can convert your module to a shared context:

07-structuring-code-examples/09/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb

```
RSpec.shared_context 'API helpers' do
  include Rack::Test::Methods

  def app
    ExpenseTracker::API.new
  end

  before do
    basic_authorize 'test_user', 'test_password'
  end
end
```

To use this shared context, you'd just call include_context from any example group that needs to make API calls:

07-structuring-code-examples/09/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb

```
RSpec.describe 'Expense Tracker API', :db do
  include_context 'API helpers'

  # ...
end
```

With include_context, RSpec evaluates your shared group block inside this group, causing it to add the hook, helper method, and module inclusion here. As with include, you can also use it in an RSpec.configure block for those rare situations where you want to include the shared group in *all* example groups: